Recursive Arithmetic

Prof. Timothy Sun

Evaluating polynomials

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

Given an array [$\boldsymbol{a}_0$, $\boldsymbol{a}_1$, ..., $\boldsymbol{a}_n$], and $\boldsymbol{x}$, how do you evaluate $\boldsymbol{p}(\boldsymbol{x})$?

Evaluating polynomials

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

Given an array [$\boldsymbol{a}_0$, $\boldsymbol{a}_1$, …, $\boldsymbol{a}_n$], and $\boldsymbol{x}$, how do you evaluate $\boldsymbol{p}(\boldsymbol{x})$?

```
NaivePoly(A[0,...,n], x):
    sum := 0
    for i = 0,...,n:
        mono := 1
        for j = 1,...,i:
            mono := mono · x
        sum := sum + mono · A[i]
    return sum
```

Evaluating polynomials

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

Given an array [$\boldsymbol{a}_0$, $\boldsymbol{a}_1$, …, $\boldsymbol{a}_n$], and $\boldsymbol{x}$, how do you evaluate $\boldsymbol{p}(\boldsymbol{x})$?

Runtime: $O(\boldsymbol{n}^2)$

```
NaivePoly(A[0,...,n], x):
    sum := 0
    for i = 0,...,n:
        mono := 1
        for j = 1,...,i:
            mono := mono · x
        sum := sum + mono · A[i]
    return sum
```

Evaluating polynomials

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

Given an array [$\boldsymbol{a}_0$, $\boldsymbol{a}_1$, …, $\boldsymbol{a}_n$], and $\boldsymbol{x}$, how do you evaluate $\boldsymbol{p}(\boldsymbol{x})$?

$\textsc{NaivePoly}(A[0,\ldots,n], x)$:
  $sum := 0$
  for $i = 0,\ldots,n$:
    $mono := 1$
    for $j = 1,\ldots,i$:
      $mono := mono \cdot x$
    $sum := sum + mono \cdot A[i]$
  return sum

Runtime: O($\boldsymbol{n}^2$)

repeated work

Horner scheme

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

Horner scheme

$$
\begin{aligned}
p(x) &= a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n \\
&= a_0 + x(a_1 + a_2 x + \ldots + a_n x^{n-1})
\end{aligned}
$$

Horner scheme

$$
\begin{aligned}
p(x) &= a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n \\
&= a_0 + x(a_1 + a_2 x + \ldots + a_n x^{n-1}) \\
&= a_0 + x(a_1 + x(a_2 + \ldots + a_n x^{n-2}))
\end{aligned}
$$

Horner scheme

$$
\begin{aligned}
p(x) &= a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n \\
&= a_0 + x(a_1 + a_2 x + \ldots + a_n x^{n-1}) \\
&= a_0 + x(a_1 + x(a_2 + \ldots + a_n x^{n-2})) \\
&= a_0 + x(a_1 + x(a_2 + \ldots + x(a_n) \ldots))
\end{aligned}
$$

Horner scheme

$$p(x) = a_0 + x(a_1 + x(a_2 + \ldots + x(a_n) \ldots))$$

$\textsc{Horner}(A[0,\ldots,n], x)$:

$\quad y := 0$

$\quad$ **for** $i = n, \ldots, 0$

$\qquad y := A[i] + xy$

$\quad$ **return** y

Horner scheme

$$p(x) = a_0 + x(a_1 + x(a_2 + \ldots + x(a_n) \ldots))$$

$$\begin{aligned}
&\textsc{Horner}(A[0,\ldots,n], x):\\
&\qquad y := 0\\
&\qquad \textbf{for } i = n,\ldots,0\\
&\qquad\qquad y := A[i] + xy\\
&\qquad \textbf{return } y
\end{aligned}$$

Runtime: O(***n***)

Recursive Horner

How about a recursive formulation?:

$$p(x) = a_0 + x(a_1 + a_2 x + \ldots + a_n x^{n-1})$$

Recursive Horner

How about a recursive formulation?:

$$p(x) = a_0 + x(a_1 + a_2 x + \ldots + a_n x^{n-1})$$

$\textsc{RecHorner}(A[0,\ldots,n], x)$:

  if $n = 0$

    return $A[0]$

  else

    return $A[0] + x \cdot \textsc{RecHorner}(A[1,\ldots,n], x)$

Fast exponentiation

Horner's scheme is optimal for ***polynomials***:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

Fast exponentiation

Horner's scheme is optimal for ***polynomials***:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

… but what about just a single large power?

$$x^N$$

Fast exponentiation

Horner's scheme is optimal for ***polynomials***:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

… but what about just a single large power?

$$x^N$$

Big exponents used in ***public-key cryptography***.

Fast exponentiation

$\Theta(\boldsymbol{N})$ is too slow:

- $\boldsymbol{N} \approx 10^{600}$

Fast exponentiation

$\Theta(\boldsymbol{N})$ is too slow:

- $\boldsymbol{N} \approx 10^{600}$, $\log_{10} \boldsymbol{N} = 600$.

$\Theta(\log \boldsymbol{N})$ is the ***number of digits*** (space) needed to write down $\boldsymbol{N}$.

Fast exponentiation

Several algorithms based on **repeated squaring**:

$$x^2 = x \cdot x$$

$$x^4 = (x^2) \cdot (x^2)$$

$$x^8 = (x^4) \cdot (x^4)$$

$$x^{16} = (x^8) \cdot (x^8)$$

Fast exponentiation

An iterative approach might look like:

- Decompose ***N*** into binary
- Multiply the “1” powers

$$x^{13} = x^{8+4+1} = x^8 \cdot x^4 \cdot x^1$$

Fast exponentiation

How would we do this recursively?

Fast exponentiation

How would we do this recursively?

$\textsc{RecPow}(x, N)$:

    if $N = 0$

        return 1

    else if $N = 1$

        return x

    else

        return $\textsc{RecPow}(x, N/2) \cdot \textsc{RecPow}(x, N/2)$

Fast exponentiation

How would we do this recursively?

$\textsc{RecPow}(x, N)$:

if $N = 0$

return 1

else if $N = 1$

return x

else

return $\textsc{RecPow}(x, N/2) \cdot \textsc{RecPow}(x, N/2)$

fails for odd numbers

Fast exponentiation

How would we do this recursively?

$\textsc{RecPow}(x, N)$:

    if $N = 0$

        return 1

    else if $N = 1$

        return x

    else

        return $\textsc{RecPow}(x, \lfloor N/2 \rfloor) \cdot \textsc{RecPow}(x, \lceil N/2 \rceil)$

Fast exponentiation

How would we do this recursively?

$\textsc{RecPow}(x, N)$:

if $N = 0$

return 1

else if $N = 1$

return x

else

return $\textsc{RecPow}(x, \lfloor N/2 \rfloor) \cdot \textsc{RecPow}(x, \lceil N/2 \rceil)$

Still O(***N***) time:

$$T(N) = 2T(N/2) + O(1) = \Theta(N)$$

Fast exponentiation

One recursive call:

```
RecPow(x, N):
    if N = 0
        return 1
    else
        p := RecPow(x, ⌊N/2⌋)
        if N is even
            return p · p
        else
            return p · p · x
```

Fast exponentiation

One recursive call:

By master theorem, runtime is

$$T(\boldsymbol{N}) = T(\boldsymbol{N}/2) + O(1) = \Theta(\log \boldsymbol{N})$$

```
RecPow(x, N):
    if N = 0
        return 1
    else
        p := RecPow(x, ⌊N/2⌋)
        if N is even
            return p · p
        else
            return p · p · x
```

Fast exponentiation

$$
\begin{aligned}
x^{13} &= (x^6)^2 \cdot x \\
&= ((x^3)^2)^2 \cdot x \\
&= (((x)^2 \cdot x)^2)^2 \cdot x
\end{aligned}
$$

```
RecPow(x, N):
    if N = 0
        return 1
    else
        p := RecPow(x, ⌊N/2⌋)
        if N is even
            return p · p
        else
            return p · p · x
```

Addition

How long does it take to add two ***n*** digit numbers together?

$$\begin{array}{r} 1\ 2\ 3\ 4\ 5\ 6\ 7 \\ +\ 7\ 1\ 5\ 8\ 3\ 0\ 9 \\ \hline \end{array}$$

Addition

How long does it take to add two ***n*** digit numbers together?

$$
\begin{array}{rccccccc}
 & & & 1 & & & 1 & \\
 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\
+ & 7 & 1 & 5 & 8 & 3 & 0 & 9 \\
\hline
 & 8 & 3 & 9 & 2 & 8 & 7 & 6
\end{array}
$$

$\Theta(\boldsymbol{n})$ operations

Multiplication?

How long does it take to multiply two ***n*** digit numbers together?

$$\begin{array}{r} 2\ 6\ 4\ 9 \\ \times\ 1\ 7\ 3\ 4 \\ \hline \end{array}$$

Multiplication?

How long does it take to multiply two ***n*** digit numbers together?

$$
\begin{array}{rrrrrrr}
 & & & 2 & 6 & 4 & 9 \\
 & & \times & 1 & 7 & 3 & 4 \\
\hline
 & & 1 & 0 & 5 & 9 & 6 \\
 & & 7 & 9 & 4 & 7 & \\
1 & 8 & 5 & 4 & 3 & & \\
2 & 6 & 4 & 9 & & & \\
\hline
4 & 5 & 9 & 3 & 3 & 6 & 6
\end{array}
$$

Multiplication?

How long does it take to multiply two ***n*** digit numbers together?

$$
\begin{array}{rrrrrrr}
 & & & 2 & 6 & 4 & 9 \\
 & & \times & 1 & 7 & 3 & 4 \\
\hline
 & & 1 & 0 & 5 & 9 & 6 \\
 & & 7 & 9 & 4 & 7 & \\
1 & 8 & 5 & 4 & 3 & & \\
2 & 6 & 4 & 9 & & & \\
\hline
4 & 5 & 9 & 3 & 3 & 6 & 6
\end{array}
$$

$\Theta(\boldsymbol{n}^2)$ operations

Karatsuba's algorithm

It is possible to do o(**n**^{2}) operations using divide-and-conquer

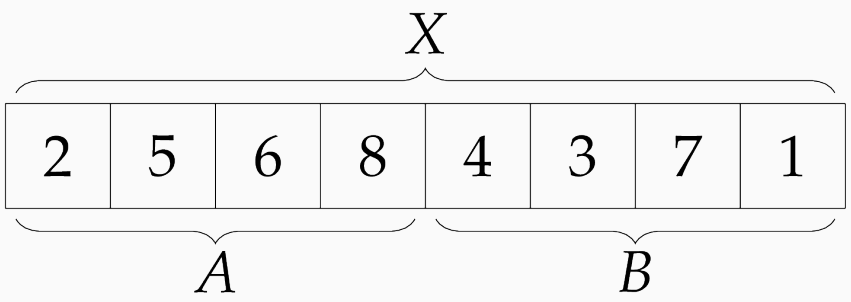
$$X \cdot Y$$

Each part ***A***, ***B***, ***C***, ***D*** has ***n***/2 digits

Karatsuba's algorithm

It is possible to do o(**n**^{2}) operations using divide-and-conquer

$$X \cdot Y$$



Each part ***A***, ***B***, ***C***, ***D*** has ***n***/2 digits

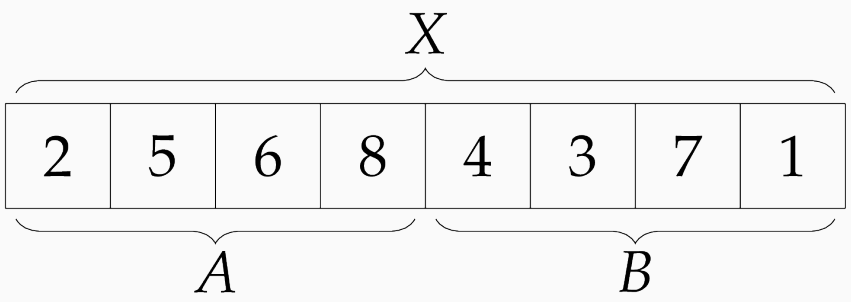
Karatsuba's algorithm

It is possible to do o(**n**^{2}) operations using divide-and-conquer

$$X \cdot Y$$

$$X = 10^{n/2}A + B$$

$$Y = 10^{n/2}C + D$$



Each part ***A***, ***B***, ***C***, ***D*** has ***n***/2 digits

A first attempt…

$$
\begin{aligned}
X \cdot Y &= (10^{n/2}A + B)(10^{n/2}C + D) \\
&= 10^n AC + 10^{n/2}(AD + BC) + BD
\end{aligned}
$$

A first attempt...

$$X \cdot Y = (10^{n/2}A + B)(10^{n/2}C + D)$$
$$= 10^n AC + 10^{n/2}(AD + BC) + BD$$

```
MultiplyDnC(X[1,...,n], Y[1,...,n]):
    if n = 1
        return X[1] · Y[1]
    else
        A := X[1,...,n/2], B := X[n/2+1,...,n]
        C := Y[1,...,n/2], D := Y[n/2+1,...,n]
        e := MultiplyDnC(A, C)
        f := MultiplyDnC(A, D)
        g := MultiplyDnC(B, C)
        h := MultiplyDnC(B, D)
        return 10^n e + 10^(n/2)(f + g) + h
```

A first attempt...

$$X \cdot Y = (10^{n/2}A + B)(10^{n/2}C + D)$$
$$= 10^n AC + 10^{n/2}(AD + BC) + BD$$

```
MultiplyDnC(X[1,...,n], Y[1,...,n]):
    if n = 1
        return X[1] · Y[1]
    else
        A := X[1,...,n/2], B := X[n/2+1,...,n]
        C := Y[1,...,n/2], D := Y[n/2+1,...,n]
        e := MultiplyDnC(A, C)
        f := MultiplyDnC(A, D)
        g := MultiplyDnC(B, C)
        h := MultiplyDnC(B, D)
        return 10^n e + 10^(n/2)(f + g) + h
```

Some helper functions

$\textsc{Add}(X[1,\ldots,n], Y[1,\ldots,n])$:

//add two n-digit numbers to get

//an array of size n or $n+1$

$\textsc{Shift}(X[1,\ldots,n], k)$:

//copy X into array of size $n+k$

//make remaining entries 0

A first attempt…

Runtime would be:

$$T(n) = 4\,T(n/2) + \Theta(n)$$

$\textsc{MultiplyDnC}(X[1,\ldots,n], Y[1,\ldots,n])$:
  if $n = 1$
    return $X[1] \cdot Y[1]$
  else
    $A := X[1,\ldots,n/2], B := X[n/2+1,\ldots,n]$
    $C := Y[1,\ldots,n/2], D := Y[n/2+1,\ldots,n]$
    $e := \textsc{MultiplyDnC}(A, C)$
    $f := \textsc{MultiplyDnC}(A, D)$
    $g := \textsc{MultiplyDnC}(B, C)$
    $h := \textsc{MultiplyDnC}(B, D)$
    return $10^n e + 10^{n/2}(f + g) + h$

A first attempt…

Runtime would be:

$$T(n) = 4\,T(n/2) + \Theta(n) = \Theta(n^2)$$

by master theorem.

$\textsc{MultiplyDnC}(X[1,\ldots,n], Y[1,\ldots,n])$:
  if $n = 1$
    return $X[1] \cdot Y[1]$
  else
    $A := X[1,\ldots,n/2], B := X[n/2+1,\ldots,n]$
    $C := Y[1,\ldots,n/2], D := Y[n/2+1,\ldots,n]$
    $e := \textsc{MultiplyDnC}(A, C)$
    $f := \textsc{MultiplyDnC}(A, D)$
    $g := \textsc{MultiplyDnC}(B, C)$
    $h := \textsc{MultiplyDnC}(B, D)$
    return $10^n e + 10^{n/2}(f + g) + h$

A first attempt…

Runtime would be:

$$T(n) = 4\,T(n/2) + \Theta(n) = \Theta(n^2)$$

by master theorem.

No improvement???

$\textsc{MultiplyDnC}(X[1,\ldots,n], Y[1,\ldots,n])$:
  if $n = 1$
    return $X[1] \cdot Y[1]$
  else
    $A := X[1,\ldots,n/2], B := X[n/2+1,\ldots,n]$
    $C := Y[1,\ldots,n/2], D := Y[n/2+1,\ldots,n]$
    $e := \textsc{MultiplyDnC}(A, C)$
    $f := \textsc{MultiplyDnC}(A, D)$
    $g := \textsc{MultiplyDnC}(B, C)$
    $h := \textsc{MultiplyDnC}(B, D)$
    return $10^n e + 10^{n/2}(f + g) + h$

Karatsuba’s algorithm

Karatsuba’s brilliant idea:

$$\boldsymbol{BC + AD} = AC + BD - (A - B)(C - D)$$

Karatsuba's algorithm

$$
\begin{aligned}
X \cdot Y &= (10^{n/2}A + B)(10^{n/2}C + D) \\
&= 10^n AC + 10^{n/2}(AD + BC) + BD \\
&= 10^n AC + 10^{n/2}(AC + BD - (A - B)(C - D)) + BD
\end{aligned}
$$

How many recursive calls now?

Karatsuba's algorithm

$$
\begin{aligned}
X \cdot Y &= (10^{n/2}A + B)(10^{n/2}C + D) \\
&= 10^n AC + 10^{n/2}(AD + BC) + BD \\
&= 10^n \boxed{AC} + 10^{n/2}(\boxed{AC} + \boxed{BD} - \boxed{(A - B)(C - D)}) + \boxed{BD}
\end{aligned}
$$

How many recursive calls now? **Looks like 5, but only 3 (2 repeated)**

Karatsuba's algorithm

$\textsc{MultiplyDnC}(X[1,\ldots,n], Y[1,\ldots,n])$:

if $n = 1$

return $X[1] \cdot Y[1]$

else

$A := X[1,\ldots,n/2], B := X[n/2+1,\ldots,n]$

$C := Y[1,\ldots,n/2], D := Y[n/2+1,\ldots,n]$

$e := \textsc{MultiplyDnC}(A, C)$

$fg := \textsc{MultiplyDnC}(A - B, C - D)$

$h := \textsc{MultiplyDnC}(B, D)$

return $10^n e + 10^{n/2}(e + h - fg) + h$

Runtime would be:

$$T(n) = \underline{3}T(n/2) + \Theta(n)$$

Karatsuba's algorithm

$\textsc{MultiplyDnC}(X[1,\ldots,n], Y[1,\ldots,n])$:

if $n = 1$

return $X[1] \cdot Y[1]$

else

$A := X[1,\ldots,n/2], B := X[n/2+1,\ldots,n]$

$C := Y[1,\ldots,n/2], D := Y[n/2+1,\ldots,n]$

$e := \textsc{MultiplyDnC}(A, C)$

$fg := \textsc{MultiplyDnC}(A - B, C - D)$

$h := \textsc{MultiplyDnC}(B, D)$

return $10^n e + 10^{n/2}(e + h - fg) + h$

Runtime would be:

$T(n) = 3T(n/2) + \Theta(n) =$

$$\Theta(n^{\log_2 3}) = O(n^{1.585})$$

by master theorem.